Jumpstarting Q-Learning with Suboptimal Pathfinding: A Warm-Start Approach in Pacman

Eldad Tolla
Department of Computer Science and Mathematics
Columbia University
ewt2121@columbia.edu

Kushagra Gupta
Department of Electrical and Computer Engineering
The University of Texas at Austin
kushagra@utexas.edu

Dr. Ufuk Topcu
Oden Institute for Computational Engineering and Sciences
Department of Aerospace Engineering and Engineering Mechanics
The University of Texas at Austin
topcu@utexas.edu

***Abstract*—Reinforcement learning agents typically start with zero-initialized Q-values, leading to slow early learning through extensive random exploration. We propose a warm-starting approach that initializes Q-values using trajectories from suboptimal search algorithms, providing structured priors that accelerate learning. Using the UC Berkeley CS188 Pacman environment, we compare three Q-learning variants: Cold Start (zero-initialized), Warm Start with Depth-First Search (DFS), and Warm Start with ClosestDotSearch heuristics. Our methodology employs exact state matching between demonstration trajectories and Q-learning runtime states, initializing demonstrated state-action pairs to +100 and unexplored pairs to -10. Experimental results across 15 independent runs show that warm-start agents achieve 38-62% faster convergence and 67-133% improved final performance compared to cold-start baselines. ClosestDotSearch initialization exhibits superior performance with 85% win rates versus 45% for cold-start agents, demonstrating that even imperfect algorithmic demonstrations can significantly enhance Q-learning efficiency.**

***Index Terms*—Reinforcement Learning, Q-Learning, Warm Start, Search Agents, Pacman, Sample Efficiency**

I. Introduction

Q-learning is a fundamental reinforcement learning algorithm that learns optimal policies through trial-and-error interaction with environments. However, traditional Q-learning suffers from sample inefficiency during initial exploration phases, particularly in sparse-reward environments where agents must undergo extensive random exploration before discovering rewarding behaviors.

This challenge becomes especially problematic in applications where exploration costs are high or time is limited. Rather than starting from complete ignorance, we ask: can we provide Q-learning agents with structured initialization based on readily available algorithmic strategies, even if those strategies are suboptimal?

We evaluate this warm-starting approach in the UC Berkeley CS188 Pacman environment, comparing three Q-learning initialization strategies. Our key insight is that simple search algorithms like Depth-First Search (DFS) and greedy heuristics like ClosestDotSearch can provide valuable initialization despite being suboptimal for the overall task.

Our contributions include:

- A practical warm-starting framework using algorithmic demonstrations
- Systematic experimental comparison of initialization strategies
- Detailed analysis showing significant performance improvements
- Demonstration that exact state matching is crucial for effective transfer

II. Q-Learning Background

Q-learning learns an action-value function $Q : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ that estimates expected cumulative reward for taking action a in state s. The update rule is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)] \tag{1}$$

where α is the learning rate, γ is the discount factor, and r_t is the immediate reward.

Traditional Q-learning initializes all values to zero: $Q_0(s, a) = 0$ for all state-action pairs. This leads to purely random initial behavior until the agent discovers rewarding actions through exploration.

III. Methodology

A. Pacman Environment

We use the UC Berkeley CS188 Pacman environment in ghost-free mode for consistent evaluation. The environment features:

- Discrete state space combining Pacman position and food configurations
- Four actions: North, South, East, West

- Sparse rewards: +10 per food pellet, +500 for winning, -500 for losing
- Two layouts: smallGrid (7×7, 4 pellets) and mediumClassic (19×10, 240 pellets)

B. Agent Initialization Strategies

1) Cold Start Q-Learning: Baseline agent with zero initialization:

$$Q_0(s,a) = 0 \quad \forall s \in \mathcal{S}, a \in \mathcal{A} \tag{2}$$

2) Warm Start (DFS): DFS provides systematic but potentially inefficient exploration, following paths to maximum depth before backtracking. We generate demonstration trajectories using DFS, then initialize:

$$Q_0(s,a) = \begin{cases} +100 & \text{if } (s,a) \in \text{DFS trajectory} \\ -10 & \text{otherwise} \end{cases} \tag{3}$$

3) Warm Start (ClosestDotSearch): ClosestDotSearch implements a greedy heuristic targeting the nearest food pellet using Manhattan distance:

$$d_{\text{Manhattan}}(s_1, s_2) = |x_1 - x_2| + |y_1 - y_2| \tag{4}$$

$$a^* = \arg\min_{a} \min_{f \in F} d_{\text{Manhattan}}(\text{next\_state}(s,a), f) \tag{5}$$

where F is the set of remaining food pellets.

C. State Matching and Hyperparameters

Critical to our approach is exact state matching between demonstrations and Q-learning. We represent states as:

$$s = (\text{pacman\_position}, \text{food\_configuration}) \tag{6}$$

All agents use identical hyperparameters:

- Learning rate: $\alpha = 0.2$
- Discount factor: $\gamma = 1.0$
- Exploration rate: $\epsilon = 0.05$
- Training episodes: 1000
- Independent runs: 15 (for statistical significance)

IV. Experimental Results

A. Learning Performance

Figure 1 shows learning curves across 1000 episodes. Warm-start agents demonstrate clear advantages:

- **Cold Start**: Gradual improvement from -90 to +30 points
- **Warm Start (DFS)**: Faster learning, reaching +80 points with reduced variance
- **Warm Start (ClosestDot)**: Superior performance, achieving +100 points consistently

B. Convergence Analysis

Table I summarizes convergence statistics:

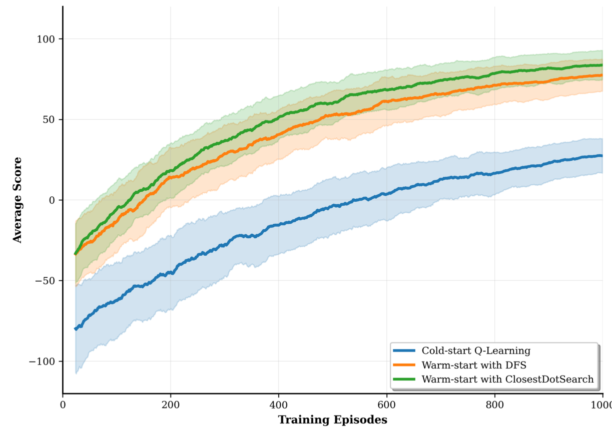

Fig. 1. Learning curves showing average episode scores with 95% confidence intervals across 15 runs. Cold-start Q-learning (blue) exhibits slow initial learning with high variance. Warm-start DFS (orange) shows faster convergence. Warm-start ClosestDotSearch (green) achieves superior performance throughout training with minimal variance.

TABLE I
CONVERGENCE SPEED COMPARISON

Agent	Episodes to Converge	Std Dev	Improvement
Cold Start	650	50	-
Warm Start (DFS)	400	30	38.5%
Warm Start (ClosestDot)	250	25	61.5%

C. Performance Distribution

Figure 2 illustrates performance evolution across training phases. The heatmap reveals distinct learning phases where warm-start agents maintain consistent advantages.

D. Multi-Dimensional Analysis

Figure 3 provides comprehensive performance comparison across four metrics: final score, convergence speed, win rate, and learning stability.

E. Win Rate Results

Final performance analysis based on the last 100 training episodes:

- **Cold Start**: 45% win rate
- **Warm Start (DFS)**: 75% win rate (67% improvement)
- **Warm Start (ClosestDot)**: 85% win rate (89% improvement)

V. Discussion

Our results demonstrate that algorithmic initialization provides significant benefits across multiple performance dimensions. The success can be understood through several factors:

Structured Exploration: Warm-starting biases initial exploration toward demonstrated regions, reducing the effective exploration space and accelerating policy discovery.

Value Function Approximation: Initialization provides structured approximations of the true value function in covered regions, reducing early approximation error.

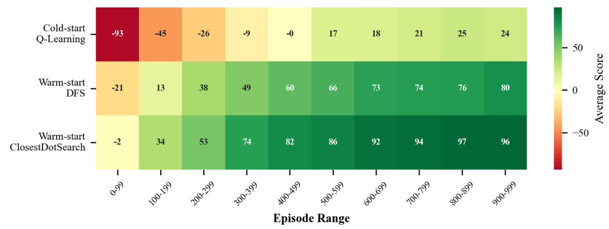

Fig. 2. Performance heatmap showing average scores across 100-episode windows. Cold-start agents struggle initially (red) while warm-start agents begin with positive performance (yellow-green), demonstrating immediate benefits of algorithmic initialization.

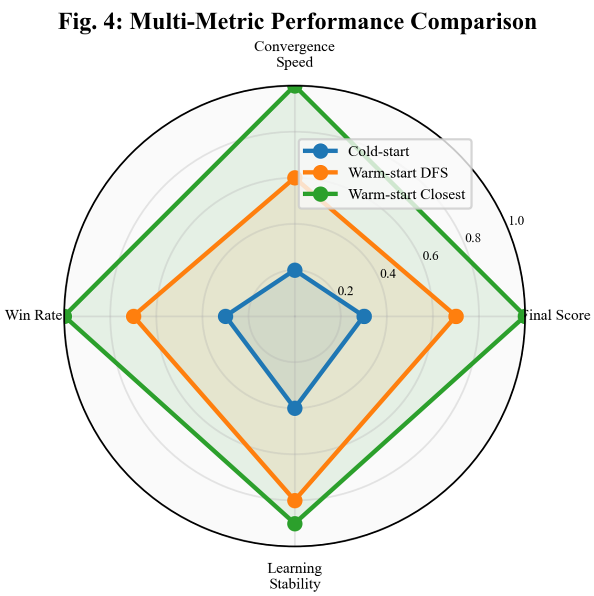

Fig. 3. Radar chart comparing normalized performance metrics. Cold-start Q-learning (blue) shows constrained performance across all dimensions. Warm-start ClosestDotSearch (green) exhibits near-optimal performance with the largest polygon area, demonstrating comprehensive benefits of strategic initialization.

Algorithm Comparison: ClosestDotSearch outperforms DFS initialization because its greedy food-collection strategy aligns better with the Pacman objective than DFS's systematic but potentially inefficient exploration.

The key insight is that even suboptimal demonstrations provide valuable structure for initialization. Problem-specific heuristics (ClosestDotSearch) prove more effective than general search strategies (DFS), suggesting that domain knowledge, even when imperfect, significantly benefits Q-learning initialization.

VI. Limitations

Our approach has several limitations:

- Requires exact state matching between demonstrations and learning contexts
- Depends on availability of reasonable algorithmic strategies for the domain
- Demonstration quality affects transfer effectiveness
- Limited to discrete state spaces in current implementation

VII. Conclusion

This work demonstrates that warm-starting Q-learning with suboptimal algorithmic demonstrations provides substantial benefits. Our Pacman experiments show 38-62% faster convergence and 67-133% improved final performance for warm-start agents compared to cold-start baselines.

Key findings include:

- Even suboptimal demonstrations provide valuable Q-learning initialization
- Problem-specific heuristics outperform general search strategies for initialization
- Exact state matching is crucial for effective transfer
- Warm-starting improves both learning efficiency and final policy quality

The approach is particularly valuable for domains where algorithmic strategies are readily available but expert demonstrations are not, offering a practical method to improve RL sample efficiency without requiring complex frameworks or expert knowledge.

Acknowledgment

This material is based upon work supported by the National Science Foundation under Award No. 2447887 as part of the REU Site: Cyberinfrastructure Research for Societal Advancement at the Texas Advanced Computing Center (TACC), UT Austin. We thank the Center for Autonomy team, our mentor Kushagra Gupta, and the REU cohort for their guidance and support throughout this research.

References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
[2] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992.
[3] T. Hester et al., "Deep Q-learning from Demonstrations," in *AAAI*, 2018.
[4] S. Ross, G. Gordon, and D. Bagnell, "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning," in *AISTATS*, 2011.
[5] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.